SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

DATA INTERPRETATION

Code of Conduct:

I KAVINMITHA(192511285)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
kavinmitha.s

Annexure A

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization
Given the three-address code:
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Q2. Symbol Table and Memory Allocation
Given the following symbol table entries:

Variable	Data Type	Size (Bytes)
p	char	1
q	double	8
r	int	4
s	float	4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Q3. Optimization in Intermediate Code
Given the intermediate code:
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Q4. Control Flow and Boolean Expression Handling
Given the Boolean expression:
if (x > y || p < q)
Intermediate code:
if x > y goto L1
if p < q goto L1
goto L2
L1: ...
Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Q5. Backpatching and Flow Control
Given:

Instruction	Targe

t

200: if x==y
goto _
201: goto _
Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

ANSWERS:

Q1. Intermediate Code Analysis and Optimization

Given:

t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p

Operation sequence:

1. t1 = x + y
2. t2 = (x + y) * z
3. t3 = ((x + y) * z) / w
4. t4 = (((x + y) * z) / w) - p

Equivalent expression:

t4=w(x+y)z−p

Expression tree:

Role of temporary variables:
Temporary variables store intermediate results such as `x+y` and `(x+y)*z`. They make the code easier to generate and optimize and avoid recalculating the same expressions.

Conclusion:
Three-address code with temporary variables provides a clear and efficient representation for compiler optimization.

Q2. Symbol Table and Memory Allocation

Given:

Variable	Data Type	Size
p	char	1 byte
q	double	8 bytes
r	int	4 bytes
s	float	4 bytes

Total memory:

1+8+4+4=17 bytes

Memory allocation:

- p → 1 byte
- q → 8 bytes
- r → 4 bytes
- s → 4 bytes

Alignment:
Different data types may require different memory boundaries. For example, a double commonly requires greater alignment than a char. Therefore, padding may sometimes be inserted between variables.

Conclusion:
The basic storage requirement is **17 bytes**, but actual memory occupied can be greater if padding is required for alignment.

Q3. Optimization in Intermediate Code

Given:

```
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
```

Step-by-step execution:

1. Calculate m * n and store it in t1.
2. Calculate p + q and store it in t2.
3. Calculate m * n again and store it in t3.
4. Subtract t3 from t2.

The expression **m * n is calculated twice**.

Optimized code:

```
t1 = m * n
t2 = p + q
t4 = t2 - t1
```

t3 is removed because t1 already contains the same value.

Optimization used:
This is **Common Subexpression Elimination** and redundant computation removal.

Conclusion:
Optimization reduces the number of operations and temporary variables, improving execution speed and efficiency.

Q4. Control Flow and Boolean Expression Handling

Given:

if (x > y || p < q)

Intermediate code:

if x > y goto L1
if p < q goto L1
goto L2
L1: ...

Control flow:

- First, the compiler checks x > y.
- If x > y is **true**, control immediately jumps to L1.
- If x > y is **false**, it checks p < q.
- If p < q is **true**, control jumps to L1.
- If both conditions are false, control goes to L2.

Short-circuit evaluation:
Because the operator is **OR (||)**, the second condition does not need to be checked when the first condition is already true.

Example:

x > y = TRUE

Then p < q is not evaluated.

Conclusion:
Short-circuit evaluation avoids unnecessary condition checking and improves execution efficiency.

Q5. Backpatching and Flow Control

Given:

Instruction	Target
200: if x==y goto _	_
201: goto _	_

Backpatch lists:

truelist = {200}
falselist = {201}

Backpatching:
Backpatching is a technique used to fill in the target addresses of jump instructions when the target location is determined later.

Suppose the true target is **205** and false target is **210**.

After backpatching:

200: if x == y goto 205
201: goto 210

So:

truelist → 200 → 205
falselist → 201 → 210

Importance:

- Helps generate correct conditional jumps.
- Supports Boolean expression translation.
- Handles forward jumps whose targets are not known initially.
- Makes control-flow generation easier.

Conclusion:
Backpatching connects incomplete jump instructions with their correct target addresses and is important for generating intermediate code for Boolean expressions and conditional statements.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)

Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand